

✓ PDS Practical-4

Name: Jafar Razzak Birbal

Div:SY-A

Roll No:A-72

Aim: Numerical Computation with NumPy: Perform different computations using NumPy library.

```
import numpy as np
```

✓ 1. Array Creation

```
# Create a simple 1D array
arr = np.array([1, 2, 3, 4, 5])
print(f"1D Array: {arr}")
print(f"Type of arr: {type(arr)}")

# Create a 2D array (matrix)
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(f"\n2D Array (Matrix):\n{matrix}")

# Create an array of zeros
zeros_arr = np.zeros((2, 3))
print(f"\nArray of Zeros (2x3):\n{zeros_arr}")

# Create an array of ones
ones_arr = np.ones((3, 2))
print(f"\nArray of Ones (3x2):\n{ones_arr}")

# Create an identity matrix
identity_matrix = np.eye(3)
print(f"\nIdentity Matrix (3x3):\n{identity_matrix}")

# Create an array with a range of values
range_arr = np.arange(0, 10, 2) # start, stop (exclusive), step
print(f"\nArray with range (0 to 10, step 2): {range_arr}")

# Create an array with linearly spaced values
linspace_arr = np.linspace(0, 1, 5) # start, stop (inclusive), number of elements
print(f"\nArray with linearly spaced values (0 to 1, 5 elements): {linspace_arr}")
```

```
1D Array: [1 2 3 4 5]
Type of arr: <class 'numpy.ndarray'>
```

```
2D Array (Matrix):
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```
Array of Zeros (2x3):
[[0. 0. 0.]
 [0. 0. 0.]]
```

```
Array of Ones (3x2):
[[1. 1.]
 [1. 1.]
 [1. 1.]]
```

```
Identity Matrix (3x3):
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
Array with range (0 to 10, step 2): [0 2 4 6 8]
```

```
Array with linearly spaced values (0 to 1, 5 elements): [0.    0.25 0.5   0.75 1.   ]
```

✓ 2. Basic Arithmetic Operations

```
arr1 = np.array([10, 20, 30, 40])
arr2 = np.array([1, 2, 3, 4])
```

```
# Addition
add_result = arr1 + arr2
print(f"Addition (arr1 + arr2): {add_result}")

# Subtraction
sub_result = arr1 - arr2
print(f"Subtraction (arr1 - arr2): {sub_result}")

# Multiplication (element-wise)
mul_result = arr1 * arr2
print(f"Element-wise Multiplication (arr1 * arr2): {mul_result}")

# Division (element-wise)
div_result = arr1 / arr2
print(f"Element-wise Division (arr1 / arr2): {div_result}")

# Exponentiation
pow_result = arr2 ** 2
print(f"Exponentiation (arr2 ** 2): {pow_result}")

# Scalar operations
scalar_add = arr1 + 5
print(f"Scalar Addition (arr1 + 5): {scalar_add}")

scalar_mul = arr2 * 3
print(f"Scalar Multiplication (arr2 * 3): {scalar_mul}")
```

```
Addition (arr1 + arr2): [11 22 33 44]
Subtraction (arr1 - arr2): [ 9 18 27 36]
Element-wise Multiplication (arr1 * arr2): [ 10 40 90 160]
Element-wise Division (arr1 / arr2): [10. 10. 10. 10.]
Exponentiation (arr2 ** 2): [ 1  4  9 16]
Scalar Addition (arr1 + 5): [15 25 35 45]
Scalar Multiplication (arr2 * 3): [ 3  6  9 12]
```

3. Array Manipulation and Statistical Operations

```
# Reshape an array
original_arr = np.array([1, 2, 3, 4, 5, 6])
reshaped_arr = original_arr.reshape((2, 3))
print(f"\nOriginal Array: {original_arr}")
print(f"Reshaped Array (2x3):\n{reshaped_arr}")

# Transpose an array
transposed_matrix = matrix.T # Using the 'matrix' from earlier
print(f"\nOriginal Matrix:\n{matrix}")
print(f"Transposed Matrix:\n{transposed_matrix}")

# Concatenate arrays
arr_concat_1 = np.array([1, 2, 3])
arr_concat_2 = np.array([4, 5, 6])
concatenated_arr = np.concatenate((arr_concat_1, arr_concat_2))
print(f"\nConcatenated Array: {concatenated_arr}")

# Statistical operations
stats_arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print(f"\nArray for statistics: {stats_arr}")
print(f"Mean: {np.mean(stats_arr)}")
print(f"Median: {np.median(stats_arr)}")
print(f"Sum: {np.sum(stats_arr)}")
print(f"Min: {np.min(stats_arr)}")
print(f"Max: {np.max(stats_arr)}")
print(f"Standard Deviation: {np.std(stats_arr)}")
```

```
Original Array: [1 2 3 4 5 6]
Reshaped Array (2x3):
[[1 2 3]
 [4 5 6]]
```

```
Original Matrix:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Transposed Matrix:
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

```
Concatenated Array: [1 2 3 4 5 6]
```

```
Array for statistics: [ 1  2  3  4  5  6  7  8  9 10]
```

```
Mean: 5.5
Median: 5.5
Sum: 55
Min: 1
Max: 10
Standard Deviation: 2.8722813232690143
```

1. Explain the role of NumPy in the Python scientific computing ecosystem.

NumPy (Numerical Python) is the fundamental package for numerical computation in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of high-level mathematical functions to operate on these arrays. It is the backbone of many other scientific computing libraries in Python, serving as the common interchange format for data.

2. Why is NumPy considered faster than pure Python computations?

NumPy achieves its speed advantage over pure Python computations primarily due to:

Vectorization: NumPy operations are implemented in C and Fortran, which are much faster than Python loops. When you perform an operation on a NumPy array, it applies the operation to all elements simultaneously (vectorization), rather than iterating through them one by one in Python. **Efficient Memory Storage:** NumPy arrays store data in a contiguous block of memory. This allows for more efficient access and manipulation of data, leveraging CPU cache better. Pure Python lists store references to objects scattered in memory, which is less efficient.

3. What types of mathematical operations can be performed using NumPy? NumPy supports a vast range of mathematical operations, including :

Basic Arithmetic Operations: Addition, subtraction, multiplication (element-wise), division, exponentiation. **Trigonometric Functions:** sin, cos, tan, arcsin, arccos, arctan, etc. **Logarithmic Functions:** log, log2, log10, exp, etc. **Statistical Operations:** Mean, median, standard deviation, variance, min, max, sum, percentile, correlation, histogram, etc. **Linear Algebra Operations:** Dot product, matrix multiplication, determinants, eigenvalues, inverse, solving linear equations. **Fourier Transforms:** fft, ifft, etc. **Random Number Generation:** Generating random numbers from various distributions. **Comparison and Logical Operations:** Element-wise comparisons (>, <, ==) and logical operations (and, or, not).

4. Explain indexing and slicing in NumPy arrays. Indexing and slicing in NumPy arrays work similarly to Python lists but with powerful extensions for multi-dimensional arrays:

Indexing: Accessing individual elements. For a 1D array, `arr[i]` gets the element at index `i`. For a 2D array, `matrix[row_index, col_index]` accesses the element at the specified row and column. Negative indices count from the end. **Slicing:** Extracting sub-arrays (views) from an array. The basic syntax is `start:stop:step`. `arr[start:stop]` selects elements from start up to (but not including) stop. `arr[start:]` selects elements from start to the end. `arr[:stop]` selects elements from the beginning up to (but not including) stop. `arr[:,step]` selects all elements with a specified step.

5. How does NumPy integrate with libraries like:

a. **Pandas:** Pandas DataFrames and Series are built on top of NumPy arrays. When you create a DataFrame, its underlying data is typically stored in one or more NumPy arrays. This allows Pandas to leverage NumPy's fast, vectorized operations for data manipulation, cleaning, and analysis. Operations on Pandas objects often result in direct calls to NumPy functions.

b. **Matplotlib:** Matplotlib, especially its pyplot module, relies heavily on NumPy arrays for plotting data. When you pass data to plotting functions like `plt.plot()`, `plt.scatter()`, or `plt.imshow()`, they expect NumPy arrays (or array-like objects that can be converted to NumPy arrays). This allows Matplotlib to efficiently handle and render large datasets.

c. **Scikit-learn:**

Scikit-learn, a popular machine learning library, uses NumPy arrays as its primary data structure for input and output. Machine learning models in scikit-learn expect features and targets to be provided as NumPy arrays (or compatible formats like Pandas DataFrames, which are converted internally). The algorithms themselves are optimized to work efficiently with NumPy's array structure, allowing for fast training and prediction on datasets.

Conclusion

NumPy plays a crucial role in numerical computation by providing efficient and fast operations on arrays and matrices. Its ability to perform vectorized calculations and handle large datasets makes it more efficient than traditional methods. By using NumPy, complex mathematical and statistical computations can be performed easily and quickly.

